

Tema 2. Diseño de bases de datos relacionales

En este tema se estudia un aspecto fundamental de las bases de datos: su diseño. En las bases de datos se ha establecido un ciclo de desarrollo que consta de tres etapas de diseño: el diseño conceptual, el diseño lógico y el diseño físico. Mientras que las dos primeras etapas y el paso de una a otra están muy fundamentados, no ocurre lo mismo con la tercera, dado que las primeras son lo suficientemente abstractas como para no depender de ninguna implementación en concreto; sin embargo, el diseño físico depende del SGBD usado, y no hay reglas formales para llevarlo a cabo.

2.1. Etapas de diseño

La metodología de diseño de bases de datos relacionales se ha consolidado a lo largo de los años satisfaciendo las propiedades de generalidad (independencia de la plataforma hardware/software), calidad del producto (precisión, completitud y eficacia) y facilidad de uso. Consta de las siguientes etapas:

1. Diseño conceptual.

   Su objetivo es definir las entidades y las relaciones entre ellos de forma abstracta, sin centrarse en ningún modelo lógico en concreto (como el relacional, el orientado a objetos, el jerárquico o el de red).

   - Herramienta: *Modelo conceptual de datos*. Se usa alguna variante del modelo entidad-relación para las bases de datos relacionales.
   - Resultado: *Esquema conceptual* de la base de datos.

2. Diseño lógico.

   Su objetivo es definir el esquema de la base de datos según el modelo que implementa el SGBD objetivo.

   - Herramienta: *Modelo lógico de datos*. Se usa el modelo lógico que implemente el sistema de gestión de bases de datos objetivo, pero es independiente de los aspectos físicos. Se usan técnicas formales para

verificar la calidad del esquema lógico; la más usual es la *normalización*. En el modelo relacional se usan las tablas.

- Resultado: *Esquema lógico* de la base de datos.

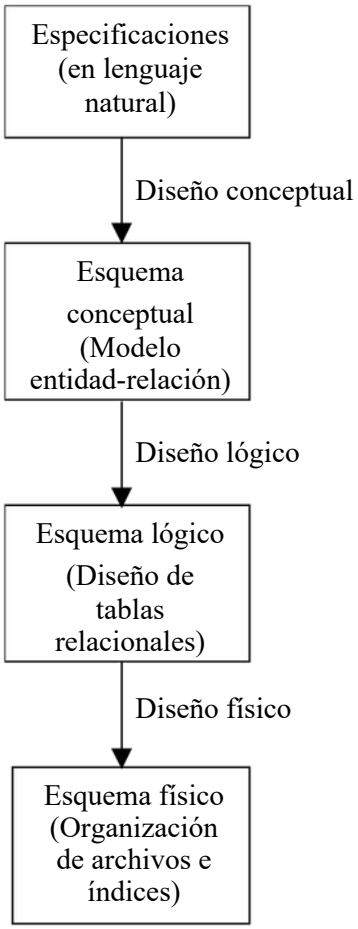
3. Diseño físico.

Su objetivo es definir el esquema físico de la base de datos de forma que se den todas las instrucciones para que un DBA pueda implementar la base de datos sin ninguna ambigüedad. Se considera el rendimiento como un aspecto que no se ha tratado en las etapas anteriores.

- Herramienta: *Modelo físico de datos*. Se consideran todos los detalles de la implementación física: organización de archivos e índices para el SGBD considerado.

- Resultado: *Esquema físico* de la base de datos.

La siguiente figura muestra resumido el ciclo de desarrollo clásico de bases de datos:



2.2. Diseño conceptual

En esteapartadose estudiael modelo entidad-relación que permite diseñar el esquema conceptual de una BD, y es muy adecuado para las BDs relacionales. Su resultado es un diagrama entidad-relación.

A lo largo de este apartado se usará un ejemplo de aplicación correspondiente a las necesidades de una secretaría de un centro docente, en la que hay alumnos matriculados en asignaturas y profesores que las imparten en ciertas aulas. Los alumnos consiguen una nota determinada en cada asignatura en que están matriculados.

2.2.1. Conceptos

• Entidad. Es el menor objeto con significado en una instancia.

Por ejemplo, para el diseño de una BD de la secretaría de un centro docente, el alumno con los siguientes datos:

DNI = 01234567Z,

Nombre y apellidos = Manuel Vázquez Prieto,

Teléfono = 91-12345678

Domicilio = Calle del Jazmín 7, 4 Izq.

constituye una entidad. Igual sucede con cada asignatura concreta, cada profesor, etc.

En el caso del enfoque "clásico" correspondería a cada registro guardado en un fichero.

- Atributo. Es cada uno de los componentes que determinan una entidad. Cada atributo tiene asociado un dominio: el conjunto de valores que puede tomar.

  La entidad del ejemplo anterior viene determinada por los valores de sus atributos DNI, Nombre y Apellidos, Teléfono, Domicilio y COU.

  En el enfoque clásico serían los campos de los registros.

- Atributos monovalorados y multivalorados. Los atributos multivalorados son los que pueden contener más de un valor simultáneamente, y monovalorados a los que sólo pueden contener un valor.

  Por ejemplo, una persona puede tener varios números de teléfono (casa, trabajo, móvil) y puede que nos interese tenerlos todos. En este caso haremos de teléfono un atributo multivalorado.

- Atributos simples y compuestos. Un atributo es compuesto cuando puede descomponerse en otros componentes o atributos más pequeños, y simple en otro caso.

  Por ejemplo, en el caso del domicilio puede que nos interese descomponerlo a su vez en calle, el número y la ciudad por separado.

- Clave. Es un atributo o conjunto de atributos cuyos valores identifican unívocamente cada entidad.

  Por ejemplo, DNI es un atributo clave del tipo de entidad Alumnos.

  Esto significa que los valores de la clave no se pueden repetir en el conjunto de entidades. En el ejemplo anterior ningún DNI se debería repetir en una instancia del tipo de entidad Alumnos.

  El concepto de clave distingue tres claves diferentes:

  - Superclave. Es cualquier conjunto de atributos que pueden identificar unívocamente a una tupla
  - Clave candidata. Es el menor conjunto de atributos que puede formar clave. Puede haber varias en una tabla.
  - Clave primaria. Es la clave candidata que distingue el usuario para identificar unívocamente cada tupla. Es importante en cuanto al aspecto del rendimiento, como se verá en el apartado dedicado al diseño físico.

- Tipo de entidad. Es el conjunto de entidades que comparten los mismos atributos (aunque con diferentes valores para ellos).

  Por ejemplo, Alumnos será un tipo de entidad que representa cualquier conjunto de entidades en el que todas tengan como atributos DNI, Nombre y Apellidos, ... y valores dentro de los dominios correspondientes. Asignaturas será otro tipo de entidad, etc.

  Intuición: En el enfoque "clásico" sería el tipo de los registros. Estamos describiendo el esquema de la base de datos.

- Relación. Es una correspondencia entre dos o más entidades. Se habla de relaciones binarias cuando la correspondencia es entre dos entidades, ternarias cuando es entre tres, y así sucesivamente.

  Por ejemplo, la relación (José García, Bases de datos) es una relación entre dos entidades que indica que el alumno José García está matriculado en la asignatura Bases de datos.

- Tipos de relación. Representan a todas las posibles relaciones entre entidades del mismo tipo.

  Por ejemplo, el tipo de relación matrícula relaciona el tipo de entidad alumnos con el tipo de entidad asignaturas.

  Observaciones:

  - Las relaciones también pueden tener atributos. Por ejemplo, Matrícula puede tener el atributo Nota que indica la nota que el alumno ha obtenido en una asignatura determinada.
  - Es posible que el mismo tipo de entidad aparezca dos o más veces en un tipo de relación. En este caso se asigna un nombre a cada papel que hace el tipo de entidad en el tipo de relación. Por ejemplo, algunos profesores tienen un supervisor, por lo que se define un tipo de

relación Supervisa que relaciona profesores con profesores, el primero tendrá el papel de supervisor y el segundo de supervisado.

2.2.2. Diagramas entidad-relación (E-R)

El diseño del modelo E-R a partir del análisis inicial no es directo. A un mismo análisis le corresponden muchos diseños "candidatos". Hay varios criterios, pero ninguno es definitivo. De un buen diseño depende:

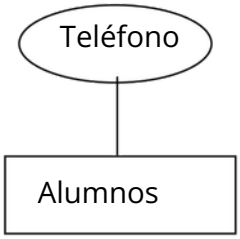
- Eficiencia: Es muy importante en las BD cuando se manejan grandes cantidades de datos.
- Simplicidad del código: Se cometen menos errores.
- Flexibilidad: Se refiere a que el diagrama sea fácil de modificar.

Los componentes básicos de los diagramas E-R son los atributos, los tipos de entidades y los tipos de relaciones.

- Tipos de entidades: Rectángulos.



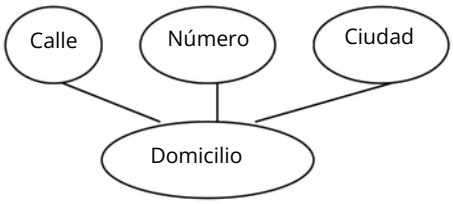
- Atributos: Elipses. Se conectan mediante líneas a los tipos de entidades o tipos de relación.



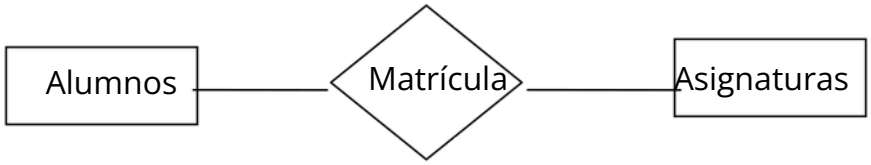
- Atributos multivalorados: Una elipse con doble línea:



- Atributos compuestos. Los componentes de un atributo se representan a su vezcomo atributos:



- Tipos de Relación: Rombos conectados a los tipos de entidades que relacionan.



2.2.3. Elección de los tipos de entidad y sus atributos

Dela especificacióndelproblema de la secretaría se deduce que va ha haber un tipo de entidad *alumnos*, pero no cuáles son sus atributos. ¿Debe incluir las asignaturas en las que está matriculado? La respuesta es no y hacerlo así sería un error grave. Aparte de la idea 'filosófica' (cada asignatura es un objeto con significado propio, es decir, una entidad), al mezclar en una sola entidad alumnos y asignaturas cometemos cuatro errores:

1. Un alumno no tiene una asignatura asociada sino un conjunto de asignaturas asociadas. En cambio, sí tiene un DNI asociado, una dirección asociada, etc. Por tanto las entidades serán de la forma

{DNI=12345678V, Nomb.Ape=Luis Martínez, Telf.=01234567, Cod=MD, Título=Matemática Discreta, Créditos=9}

{DNI=12345678V, Nomb.Ape=Luis Martínez, Telf.=01234567, Cod=IS, Título=Ingeniería del Software, Créditos=X}

{DNI=12345678V, Nomb.Ape=Luis Martínez, Telf.=01234567, Cod=LPI, Título=Laboratorio de programación I, Créditos=X}

Hay redundancia en la información de alumnos: se repite en cada entidad.

2. Las asignaturas son siempre las mismas, con lo que por cada alumno que se matricula en la misma asignatura hay que repetir toda la información:

{DNI=12345678V, Nomb.Ape=Luis Martínez, Telf.=01234567, … ,

Asignaturas={ {Cod=MD, Título=…}, {COD=IS,Título=…}, {Cod=LPI,Título=…} } }

{DNI=0000001, Nomb.Ape=Eva Manzano, Telf.=01234567, … ,

Asignaturas={ {Cod=MD, Título=…}, {COD=IS,Título=…}, {Cod=BDSI,Título=…} } }

En este caso hay redundancia en la información de las asignaturas).

3. Por cada profesor hay que apuntar las asignaturas que imparte. La información de las asignaturas debe estar por tanto relacionada con la de los profesores, pero ya está incluida con los alumnos. Hay que repetir la información de las asignaturas por lo que se consigue más redundancia.

4. No se pueden guardar los datos de una asignatura hasta que no se matricule un alumno en ella. Puede ser que en secretaría quieran meter los datos de las asignaturas antes de empezar el proceso de matrícula: No pueden. Una solución sería incluirlos con los datos de los alumnos

vacíos (nulos), lo cual no sería nada aconsejable. Los valores nulos se deben evitar siempre que sea posible.

Por tanto, hay que distinguir entre el tipo de entidad *Alumnos* y el tipo de entidad *Asignaturas*. Ambas se relacionarán mediante un tipo de relación *Matrícula.* Los restantes tipos de entidad serán: *Profesores* y *Aulas*.

Los atributos de cada tipo de entidad:

• Alumnos: DNI, Apellidos y Nombre, Domicilio, Teléfono y COU

• Asignaturas: Código, Título, Créditos

• Profesores: DNI, Apellidos y nombre, Domicilio y Teléfono

• Aulas: Edificio y Número

Aún nos falta un atributo, que es la *nota*: ¿Dónde se coloca? En Alumnos no porque un alumno tiene muchas notas, tantas como asignaturas en las que esté matriculado. En Asignaturas no porque en la misma asignatura están matriculados muchos alumnos. Va a ser un atributo del tipo de relación *matrícula*.

2.2.4. Elección de los tipos de relación

El primer tipo de relación es *Matrícula* que relaciona cada alumno con las asignaturas en las que está matriculado. Además, está relación tiene un atributo, *nota*, que se asocia cada tupla de la relación. El segundo tipo de relación es *Supervisa* que va de *Profesores* a *Profesores* y que incluye los papeles *Supervisor* y *Supervisado*. La última es *Imparte*, que relaciona cada profesor con la asignatura que imparte y el aula en la que da esa asignatura. Aquí también surgen varias posibilidades:

a) Hacer dos relaciones binarias. Por ejemplo, profesor con asignatura y asignatura con aula.

b) Hacer una relación ternaria entre profesores, aulas y asignaturas.

c) Hacer tres relaciones binarias Profesores-Asignaturas, Profesores-Aulas, Asignaturas-Aulas.

Diferencias:

1. En las opciones a) y c) se permite que un profesor imparta una asignatura que aún no tiene aula (esto puede ser deseable o no).

2. El problema de a) es:

Profesores-Asignaturas = { ({DNI=6666666, NombreYApe=Rómulo Melón},{Cod=MD,....}) ...}

Asignaturas-Aulas = { ({Cod=MD....},{Edif=Mates, NumAula=S103}),

({Cod=MD....},{Edif=Biológicas, NumAula=104}) }

Estas relaciones son posibles, hay más de un curso de primero y por eso la misma asignatura se imparte en varias aulas. Ahora bien, Don

Rómulo quiere saber en qué aula debe dar su clase de Matemática discreta, y para ello pregunta en secretaría. ¿Qué se le contesta?

Sin embargo, con la propuesta b) sí se le puede asignar a cada profesor un aula concreta para cada una de sus asignaturas.

El problema de c) sigue siendo el mismo:

Profesores-asignaturas = { ({DNI=6666666, NombreYApe=Rómulo Melón,...},{Cod=MD,....}),

({DNI=6666666, NombreYApe=Rómulo Melón,...},{Cod=IS,....}),...}

Asignaturas-Aulas =

{ ({Cod=MD....}, {Edif=Mates, NumAula=S103}),

({Cod=MD....}, {Edif=Biológicas, NumAula=104}),

({Cod=IS....}, {Edif=Mates, NumAula=S103}),

({Cod=IS....}, {Edif=Biológicas, NumAula=104}) }

Profesores-Aulas =

{

({DNI=6666666, NombreYApe=Rómulo Melón,...}, {Edif=Mates, NumAula=S103}),

({DNI=6666666, NombreYApe=Rómulo Melón,...}, {Edif=Biológicas, NumAula=104}),

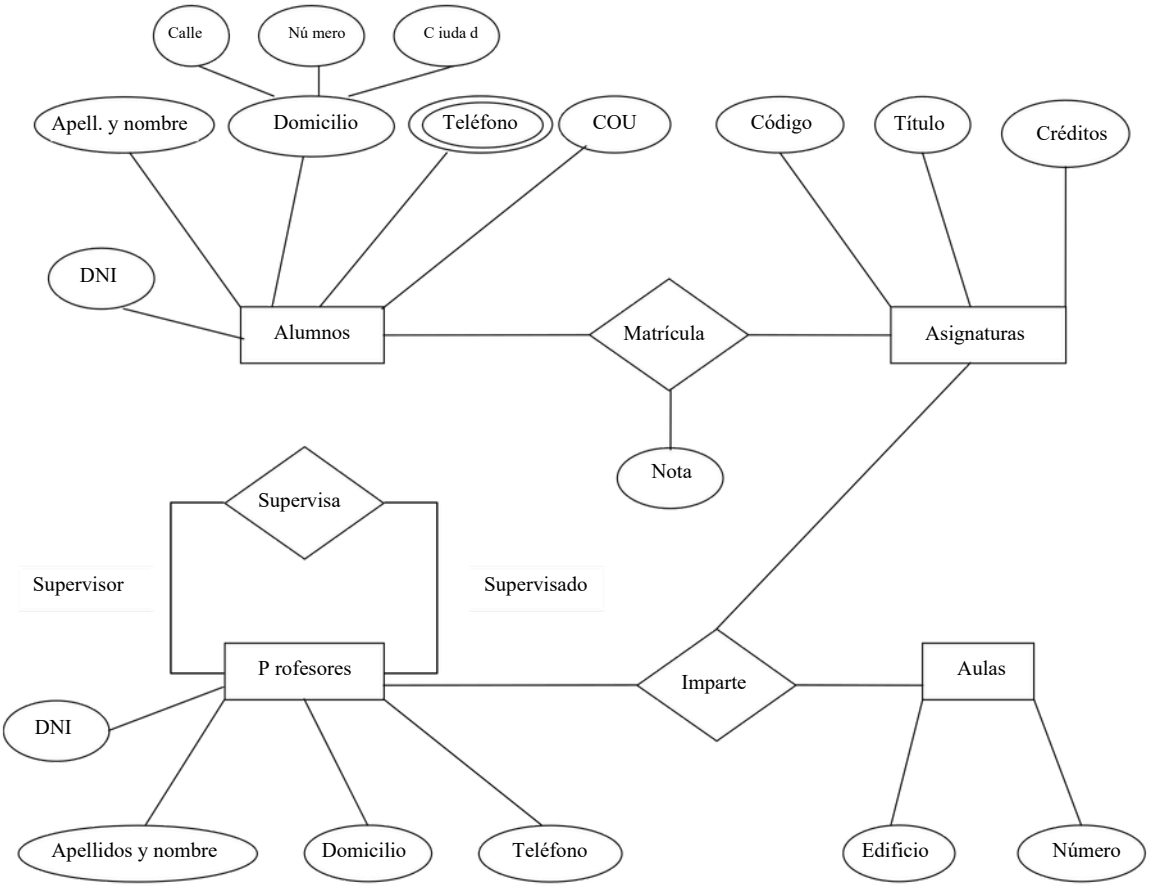
...}

Don Rómulo sabe que da 2 asignaturas, cada una en un aula, pero sigue sin saber a dónde tiene que ir a dar MD.

Conclusión:

Una relación ternaria tiene en general más información que 3 binarias.

El diagrama entidad-relación del ejemplo quedaría como se ilustra a continuación:



2.2.5. Adelanto de las restricciones de integridad

Conloselementos anteriores tenemos una primera aproximación a los diagramas E-R en la que tenemos definidos los elementos principales de los diagramas. Sin embargo, en el modelo E-R también se pueden definir numerosas *restricciones de integridad* sobre los tipos de entidades y tipos de relaciones.

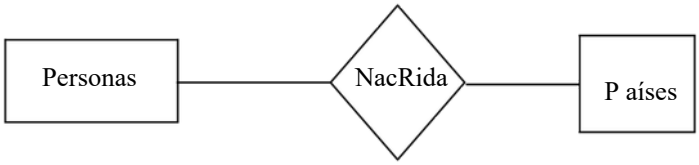
Por ejemplo, en la relación *Supervisa* un profesor puede tener a lo sumo un supervisor, pero el diagrama anterior permite

SUPERVISOR	**SUPERVISADO**
({DNI=666666,...},	{DNI=444444,...})
({DNI=000001,...},	{DNI=444444,...})

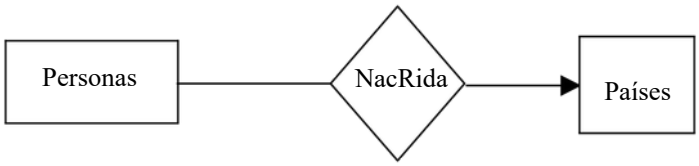
Que no debería ser una instancia válida de la relación. Para evitar estas situaciones se introducen las denominadas restricciones de integridad de la base de datos. Las *restricciones de integridad* (o simplemente restricciones) son propiedades que se asocian a un tipo de entidad o de relación. Las *instancias válidas* del tipo de entidad o relación son aquellas en las que se verifique el conjunto de restricciones asociadas. Las restricciones son parte del diseño de la base de datos igual que los tipos de entidades o de relaciones. Los SGBD se encargan de

comprobar que la instancia verifica las restricciones más usuales. En el ejemplo anterior, una vez incluida la restricción, el SGBD no nos permitiría insertar la segunda fila.

Un tipo de restricción de integridad que interesa conocer en esta etapa es la restricción de *clave*. Una restricción de clave consiste en imponer que un conjunto de atributos sea el que defina unívocamente a una fila de un tipo de entidades. Por ejemplo, en el tipo de entidades Alumnos se puede elegir DNI para identificar a un alumno en concreto, pero no sería conveniente usar el atributo Nombre y apellidos porque es muy posible encontrar a dos personas con los mismos nombres y apellidos. Por motivos de eficiencia conviene que el número de atributos elegidos sea el menor posible. A veces, es posible elegir varios conjuntos de atributos que contengan el mismo número de atributos, pero se suele escoger uno de estos conjuntos como el representativo, que se denomina *clave primaria*. Por ejemplo, si hubiese un atributo Número de la Seguridad Social, se podría usar también como clave. Todos estos conjuntos con el menor y mínimo número de atributos se denominan colectivamente como *claves candidatas*.
Otro tipo de restricción de integridad importante ahora son las restricciones de *cardinalidad*. La idea de este tipo de restricción se puede entender con el siguiente ejemplo: supongamos que deseamos tener información sobre el país de nacimiento de personas. Habría una relación Nacida entre las entidades Personas y Países, como se muestra a continuación:

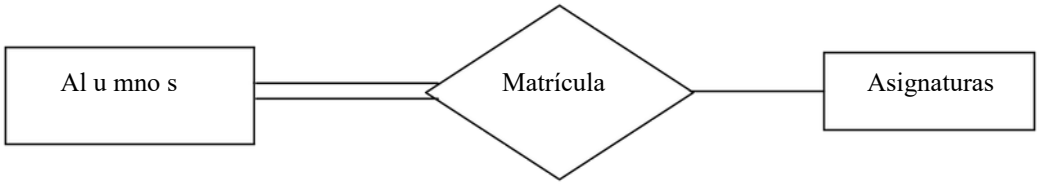


Además, deseamos expresar que, si bien muchaspersonas pueden haber nacido en un país, una persona en concretosólo puede haber nacido en un país. Esto se expresa en el diagrama E-Rconuna flecha que indica que una persona ha nacido en un país en concreto.Leyendo la relación en el sentido contrario diríamos que en un país puedenhaber nacido muchas personas (el segmento que une Nacida con personasno lleva flecha). (Esta restricción de cardinalidad también la podemosencontrar en el ejemplo de la secretaría, en la relación Supervisa, como sehavisto en el inicio de este apartado).



El último tipo de restricción de integridadque interesa introducir ahora es la *participación total*. Se refiere a quepodamos encontrar cada entidad de un tipo de entidad en la relación que loliga con otro u otros. Por ejemplo, en la base de datos de la secretaría, si hayun alumno en concreto dado de alta en la base de datos es porque se debehaber matriculado de

alguna asignatura. Es decir, cada alumno definido en el tipo de entidad Alumnos debemos encontrarlo en la relación Matrícula, relacionado con la asignatura en la que esté matriculado. En el diagrama E-R se expresa con una línea doble, como se ve a continuación:



2.3. Diseño lógico

El diseño lógico es la segunda etapa del diseño de bases de datos en general y de las bases de datos relacionales en particular. En nuestro caso, las BD relacionales, el resultado de esta etapa es un esquema relacional basado en un modelo relacional. En este apartado se describirá en primer lugar el modelo relacional y en segundo lugar cómo pasar de un esquema entidad-relación a un esquema relacional.

2.3.1. El modelo relacional

Este modelo fue creado por Codd a principios de los 70 al que dotó de una sólida base teórica. Actualmente está implementado en la mayoría de los SGBD usados en la empresa. El concepto principal de este modelo es la *relación* o *tabla*. Es importante no confundir la tabla con las relaciones del modelo E-R. Aquí las relaciones se aplican tanto a tipos de relaciones como a tipos de entidades. En este modelo no se distingue entre tipos de entidades y tipos de relaciones porque la idea es que una relación o tabla expresa la relación entre los tipos de valores que contiene.

A continuación se introducen los conceptos de este modelo:

• Entidad. Igual que en el modelo E-R. También se les llama *tuplas o filas* de la relación.

• Atributo. Igual que en el modelo E-R. También se le llaman *campos* o *columnas* de la relación. El dominio de los atributos tiene que ser simple: no se admiten atributos multivalorados ni compuestos.

• Esquema de una relación. Viene dado por el nombre de la relación y una lista de atributos. Es el tipo de entidad.

• Conjunto de entidades. *Relación* o *tabla*.

Por ejemplo, el tipo de entidad Alumnos del modelo E-R del apartado del diseño conceptual se representaría como la siguiente relación:

Alumnos(DNI, NombreYApellidos, Domicilio, Teléfono, COU)

El orden de los atributos en la lista no importa. Lo fijamos porque nos viene bien para representarlo como tabla, pero cualquier permutación es válida.

• Clave. Igual que en el modelo E-R. Hay que darse cuenta que en el modelo relacional todas las tablas deben tener claves, y que algunas tablas van a representar relaciones del esquema E-R.

• Instancia de una relación. Son conjuntos de entidades. Cada entidad se representa como una tupla. Cada componente de la tupla corresponde con el valor del atributo correspondiente, según el orden enunciado en el esquema de la relación.

Por ejemplo, una instancia de la relación Alumnos sería:

{ (01234567Z, Manuel Vázquez Prieto, Calle del Jazmín 7 4 Izq, 91-12345678, COU = SÍ), ...}

En el modelo relacional no se representan diagramas del esquema de la BD. Por el contrario, el esquema relacional se representa por los conjuntos de entidades como hemos visto antes (nombre de la tabla y entre paréntesis el nombre de sus atributos). Las *instancias* de una relación se representan con tablas, como se muestra en el ejemplo del conjunto de entidades Alumnos:

DNI	***Nombre y Apellidos***	***Domicilio***	***Teléfono***	***COU***
01234567Z	*Manuel Vázquez Prieto*	*Calle del Jazmín 7 4 Izq*	*9112345678*	*SI*
...				

2.3.2. Paso de un esquema E-R a un esquema relacional

A continuación se describe la forma de traducir cada uno de los elementos que aparecen en el modelo E-R a un esquema relacional.

Tipos de entidades

Para cada tipo de entidad se crea una relación con el mismo nombre y conjunto de atributos. Por ejemplo, en el caso de la BD de secretaría los tipos de entidades dan lugar a las siguientes relaciones:

Alumnos(DNI, Apellidos y Nombre, Domicilio, Teléfono, COU)

Asignaturas(Código, Título, Créditos)

Profesores(DNI, Apellidos y nombre, Domicilio, Teléfono)

Aulas(Edificio, Número)

Tipos d e relaciones

Para cada tipo de relación *R* se crea una relación que tiene como atributos:

• Los atributos de la clave primaria de cada tipo de entidad que participa en la relación.

• Los atributos de la propia relación.

En ocasiones hay que renombrar atributos para evitar tener varios con el mismo nombre.

Como ejemplo, en el caso de la BD de secretaría los tipos de relación dan lugar a las siguientes relaciones:

Matrícula(DNI, Código, Nota)

Supervisa(DNISupervisor, DNISupervisado)

Imparte(DNI, Código, Edificio, NumAula)

Obsérvese que en la relación Supervisa se han renombrado los atributos DNI para indicar el papel de cada uno de ellos en la relación y además evitar que se use el mismo nombre para más de un atributo,

Claves

Hay dos casos:

1. La relación proviene de un *tipo de entidad* en el esquema E-R. La clave es la misma que la del tipo de entidad. Por ejemplo:

   Alumnos(DNI, Apellidos y nombre, Domicilio, Teléfono, COU)

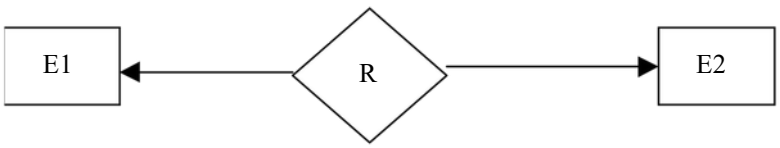
2. La relación proviene de un *tipo de relación* en el esquema E-R. Si la relación R es un tipo de relación entre varios tipos de entidades se va a construir una relación bajo el modelo E-R a partir de R con los atributos que forman clave primaria en todas las entidades participantes más los propios de R. De ellos formarán clave primaria las claves primarias de cada una de las entidades participantes. Por ejemplo:

   Matrícula(DNI, Código, Nota)

Restricciones de cardinalidad

Es posible incorporar este tipo de restricciones de integridad cuando se desean indicar relaciones una a una, una a varias y varias a varias. (En el ejemplo de la secretaría tenemos la relación Supervisa, del tipo una a varias.) A continuación se muestran estos casos para relaciones binarias, siendo c1 y c2 las claves primarias de E1 y E2, respectivamente:

a) Una a una



De los atributos de la traducción de R almodelo relacional podemos escoger como clave o bien c1 o bien c2,deellas la más adecuada será la que tenga menos atributos. Estoesposible porque cada entidad de E1 está relacionada con sólo unadeE2y viceversa, por lo que no es posible que la misma entidad de E1odeE2 aparezca más de una vez en R. Por tanto, cualquiera de susclaves primarias puede ser clave de R.

HombrPeosr(DNIe)=je{1m,2p}l,o M, ujesru(pDoNnI)g=a{m3,4o}s, la qruelea ción Casado(DNIH, DNIM)
sólo puede contener {(1,3), (2,4)} o {(1,4), (2,3)} o, en representación tabular:

Hombres

DNI 1 2

Mujeres

DNI 3 4

Casados

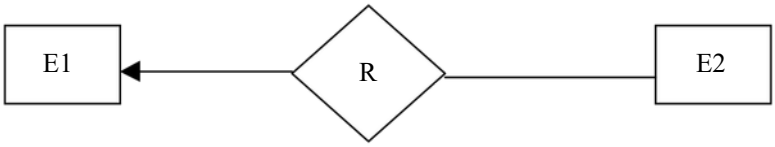
DNIH	**DNIM**
1	3
2	4

Casados

DNIH	**DNIM**
1	4
2	3

Dado que no se repite ningún valor de las columnas de las dos posibilidades de R, tanto A como B podrían ser clave. Así, tendríamos las alternativas siguientes para la relación Casados: Casados(DNIH, DNIM) y Casados(DNIH, DNIM).

b) Una a varias



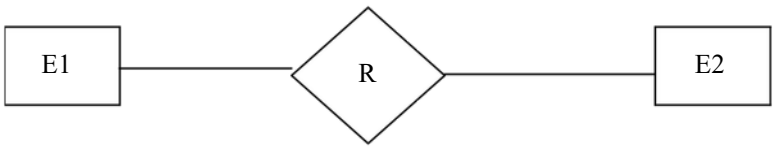
En este caso, la clave de R debe ser la claveprimaria de c2. Es decir, en la relación R no puede aparecer repetidoningún valor de E2. En el ejemplo de las personas nacidas en países,tendríamos una instancia de la relación Nacida:

IDPersona 1 2 3

	País
	España
	España
	Portugal

en donde vemos que los valores de los países se pueden repetir en la relación, pero no el identificador de la persona porque, si así fuese, significaría que la misma persona ha nacido en diferentes países. Otro ejemplo es el de la secretaría, en el que la relación Supervisa quedaría Supervisa(DNISupervisor, DNISupervisado).

c) Varias a varias



Éste es el caso más general en el quenose puede imponer ninguna restricción además de la ya indicada declave. Por ejemplo,

Los tres casos anteriores se referían a relacionesbinarias. En el caso de que se trate de relaciones n-arias, supongamos quelarelación proviene de un tipo de relación R entre tipos de entidad E1, E2,...,Ek, entonces:

- Si todos participan con cardinalidad *varios* en R, entonces una clave es la unión de las claves de E1, E2, ..., Ek.

- Si algunos tipos de entidad participan con cardinalidad *una* en R, entonces uno de ellos puede ser excluido de la superclave.

2.3.3. Restricciones de integridad

Hemos visto cómo se traducen las claves de los tipos de entidades y cómo aparecen claves en la traducción de los tipos de relaciones. Sin embargo, no es el único tipo de restricciones de integridad que aparece automáticamente al traducir un esquema E-R en otro relacional. Hay dos: restricciones de integridad referencial y restricciones de participación total. Las claves y las restricciones de integridad referencial son características que se expresar directamente en la práctica totalidad de los SGBD relacionales. Estos sistemas se ocupan automáticamente de que no se violen estas restricciones. Sin embargo, no ocurre lo mismo con las de participación total y otras restricciones, como se verá en el tema dedicado a las restricciones de integridad.

Restricciones de integridad referencial

Al traducir un tipo de relación R, en cualquier instancia de R se debe cumplir que los valores de los atributos que hereda de una entidad (de su clave primaria) deben aparecer previamente en el conjunto de entidades. En el ejemplo de los hombres y mujeres casados está claro que en la relación Casados no puede aparecer un valor del DNI de un hombre o de una mujer que no estén previamente en el conjunto de entidades Hombres o Mujeres. Es decir:

Hombres

DNI 1 2

Mujeres

DNI 3 4

Casados

DNIH	**DNIM**
1	***5***
2	4

¡Incorrecto!

En esta instancia se estaría dando la idea de que el hombre con DNI 1 está casado con la mujer con DNI 5. Pero no sabemos nada de esta mujer dado que no está en el conjunto de entidades Mujeres (hay que considerar que este tipo de entidad tendría otros muchos atributos, como el nombre y apellidos de la mujer, su domicilio, etc., que podrían ser útiles para la aplicación de la base de datos). No obstante, lo que sí es posible que algunos hombres o algunas mujeres no estén casados entre sí.

Es necesario, por tanto imponer una restricción de integridad referencial entre los atributos clave heredados de una entidad con los clave de esa entidad. En este ejemplo, se podría expresar:

Casados.DNIH $\subseteq$ Hombres.DNI

Casados.DNIM $\subseteq$ Mujeres.DNI

Es decir, los valores de DNI que aparecen en el atributo DNIH de Casados deben aparecer previamente en el atributo DNI de Hombres (y lo análogo para las mujeres).

Restricciones de participación total

Cuando cada valor de un tipo de entidad debe aparecer en un tipo de relación, como Alumnos en Matrícula, significa que, además de la restricción de integridad referencial comentada en el apartado anterior, se debe cumplir que todo valor de DNI en Alumnos debe aparecer en el atributo DNI de Matrícula. Esto se puede expresar:

Alumnos.DNI ⊆ Matrícula.DNI

Por otra parte, dado que la restricción de integridad referencial sobre esta tabla arroja:

Matrícula.DNI ⊆ Alumnos.DNI

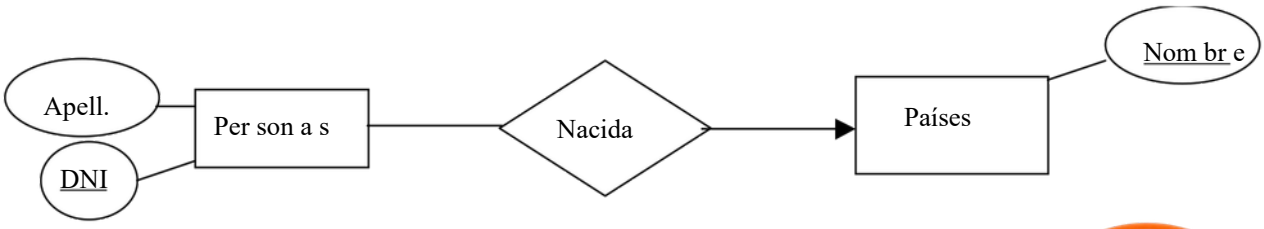
Llegamos a la conclusión de que:

Matrícula.DNI = Alumnos.DNI

Es decir, si aparece un valor de DNI en Matrícula, también debe aparecer en el atributo DNI de Alumnos, y viceversa.

2.3.4. Cuestiones de diseño

En ocasiones es posible combinar dos o más tablas en una sola. Generalmente se combinan por motivos de rendimiento. Por ejemplo, dado el ejemplo de personas nacidas en países:



La traducción de este esquema E-R al relacional sería:

Personas(DNI, Apell.)

Países(Nombre)

Nacida(DNI, Nombre)

Y se podrían combinar las dos primeras tablas en el nuevo esquema:

Personas(DNI,Apell, PaísNac)

Países(Nombre)

Un inconveniente de esta combinación es que, dado que no se exige participación total de Personas en Nacida, no tendremos información del país de nacimiento de algunas personas, y en la tabla Personas va a aparecer un valor NULL (nulo) en el atributo PaísNac, que indica que no se dispone de esa información.

El valor NULL es un valor que puede contener cualquier atributo, y lo soportan todos los SGBD. Es un valor especial que se debe tratar con

cuidado y, en general, evitar, porque puede representar muchas cosas, tales como:

- Ausencia de información.
- Este atributo no se aplica o no tiene sentido para esta entidad en
- concreto.

  Valor desconocido.

Además causan problemas a la hora de realizar consultas sobre la base de datos. Por otra parte, ningún atributo que forme parte de una clave puede tomar el valor NULL.

2.4. Diseño físico

El objetivo del diseño físico es la generación del esquema físico de la base de datos en el modelo de datos que implementa el SGBD. Esto incluye la definición sobre el SGBD de las tablas con sus campos, la imposición de todas las restricciones de integridad y la definición de índices.

Los índices son estructuras de datos implementadas con ficheros que permiten un acceso más eficaz a los datos. Se organizan con respecto a uno o más campos (los denominados campos clave del índice, que no hay que confundir con el concepto de clave del modelo entidad-relación y relacional) y guardan sólo la información del valor de la clave y la dirección física a partir de la cual se pueden encontrar registros con ese valor.

Los índices son secuencias de registros que tienen dos campos: el valor de la clave y la dirección física del registro del fichero de datos en donde se puede encontrar una tupla con ese valor.

El rendimiento (el tiempo que se tarda en realizar una determinada operación) de una base de datos depende fundamentalmente de las operaciones de lectura y escritura en disco. Como las tablas generalmente no caben todas en la memoria principal, en general es necesario leer los datos del disco. Cuantos menos datos se lean o escriban en disco mejor será el rendimiento. Los índices permiten disminuir el tiempo de entrada/salida a disco.

Internamente, cuando el SGBD necesita buscar un registro según un valor de un campo (por ejemplo, un número de DNI) sobre el que se ha definido un índice para resolver alguna consulta, busca el valor en el índice, consulta la dirección del registro adjunto y a continuación busca en el fichero de datos (donde se almacenan los datos de la tabla correspondiente) el registro.

Esto es más rápido que hacer una búsqueda secuencial en el fichero de datos. Por un lado porque los índices no requieren en general una exploración secuencial de sus registros hasta encontrar el valor deseado, sino que se organizan como estructuras que permiten localizar el valor en menos tiempo. Por otro lado, cuando se recorre el índice se hace sobre registros pequeños, en comparación con los registros más grandes que

contiene el fichero de datos; por lo tanto, el número de accesos a disco es menor.

No obstante, todo tiene un precio. Si se declara un índice, ese índice se debe mantener actualizado cada vez que la tabla sufra cualquier modificación. Si se definen muchos índices, el rendimiento de las actualizaciones se puede reducir significativamente. Por otra parte, si hay alternativas, siempre es mejor definir índices para los campos de menor tamaño, ya que cuanto más pequeño sea el campo clave, más pequeño será el índice y se necesitarán menos operaciones de lectura del índice.

Los índices se pueden definir como únicos o no (es decir, con duplicados o sin ellos). Los índices únicos indican que se aplican sobre campos en los cuales no debe haber elementos repetidos. Todas las claves primarias llevan asociado un índice de forma predeterminada. También se puede indicar que acepten valores nulos o no. Si se aceptan, el índice permitirá esos valores nulos, pero los registros que los contengan no estarán apuntados por el índice.

2.5. Restricciones de integridad

En este tema se trata uno de los aspectos más importantes para añadir consistencia a los diseños de bases de datos: son las restricciones de integridad que ayudan a mantener la consistencia semántica de los datos. Además de las restricciones de integridad definidas por las claves, las restricciones de cardinalidad y las de participación total estudiadas anteriormente, se tratan las restricciones de los dominios, la integridad referencial, las dependencias funcionales y las dependencias multivaloradas.

Las restricciones de integridad proporcionan un medio de asegurar que las modificaciones hechas a la base de datos por los usuarios autorizados no provoquen la pérdida de la consistencia de los datos. Protegen a la base de datos contra los daños accidentales (no contra daños intencionados, de lo cual se ocupa la seguridad de las bases de datos). Los tipos de restricciones de integridad en una base de datos se pueden resumir como sigue:

- Claves.
- Cardinalidad de la relación.
- Restricciones de los dominios.
- Integridad referencial.
- Participación total.
- Dependencias funcionales.
- Otras restricciones.

En este último caso se recogen las restricciones que no se pueden catalogar en los casos anteriores. Por ejemplo, si se tiene una tabla en la que se describan un compuestos químicos y sus componentes, sería deseable

imponer que la suma de los componentes, expresados en porcentajes, fuese el 100 por cien.

En las bases de datos tenemos los siguientes mecanismos para implementar las restricciones de integridad:

- Declaración de claves.
- Declaración de integridad referencial.
- Declaración de tipo de cardinalidad.
- Disparadores (Triggers).

2.5.1. Restricciones de los dominios

Las restricciones de los dominios son la forma más simple de restricción de integridad. Se especifica para cada atributo un dominio de valores posibles. Una definición adecuada de las restricciones de los dominios no sólo permite verificar los valores introducidos en la base de datos sino también examinar las consultas para asegurarse de que tengan sentido las comparaciones que hagan. Por ejemplo, normalmente no se considerará que la consulta "Hallar todos los clientes que tengan el nombre de una sucursal" tenga sentido. Por tanto, *nombre-cliente* y *nombre-sucursal* deben tener dominios diferentes.

La cláusula **check** de SQL:1999 permite restringir los dominios de maneras poderosas que no permiten la mayor parte de los sistemas de tipos de los lenguajes de programación. La cláusula **check** permite especificar un predicado que debe satisfacer cualquier valor asignado a una variable cuyo tipo sea el dominio. Por ejemplo:

Número de cifras
Número de decimales

create domain *sueldo-por-hora* **numeric**(4,2)
constraint *comprobación-valor-sueldo* **check(value** ≥ 6.00)

Restricciones de existencia

Dentro de las restricciones de los dominios, un tipo especial de restricción que se puede aplicar a cualquier dominio es la restricción de existencia. Esta restricción evita la aparición de valores *nulos* en las columnas. Se especifica indicando en la creación de la tabla cuáles son los matrainbeuctuae s no pueden contener valores nulos. De predeterminada, los atributos que formen parte de la clave primaria tienen esta restricción impuesta. Para declarar esta restricción en la definición de la tabla se usaría NOT NULL después del nombre del atributo y su dominio.

Restricciones de unicidad

Otro tipo especial de restricción que se puede aplicar a cualquier dominio es la restricción de unicidad. Esta restricción evita la aparición de

valores *duplicados* en las columnas (de forma parecida a lo que hace la clave primaria). Por ejemplo:

Sólo se admite una sucursal en cada ciudad.

```
CREATE TABLE Sucursales
(nombre-sucursal VARCHAR(20),
 ciudad-sucursal VARCHAR(20) NOT NULL,   -Restricción de existencia
PRIMARY KEY(nombre-sucursal)
UNIQUE (ciudad-sucursal))                -Restricción de unicidad
```

2.5.2. Restricciones de integridad referencial

La *integridad referencial* permite asegurar que un valor que aparece en una relación para un conjunto de atributos determinado aparezca también en otra relación para ese mismo conjunto de atributos.

Este tipo de restricciones se denota simplificadamente:

Relación1.(Atributo1-1,...,Atributo1-N) ⊆

Relación2.(Atributo2-1,...,Atributo2-N)

El conjunto de atributos {Atributo1-1,...,Atributo1-N} se denomina clave externa, mientras que el conjunto de atributos {Atributo2-1,...,Atributo2-N} es la clave primaria de Relación2.

Ya se ha visto cómo aparece este tipo de restricción de integridad en la traducción del esquema E-R al relacional. El sistema, por su parte, puede asegurar la imposición de estas restricciones de integridad, evitando la aparición de valores que las violen.

En concreto, la *modificación* de la base de datos puede ocasionar violaciones de la integridad referencial. Se distinguen tres casos:

• Al insertar una tupla es necesario comprobar que haya otra con los valores de la clave externa igual a los de sus atributos clave.

• Al borrar una tupla de R el sistema debe calcular el conjunto de tuplas de las otras relaciones que hacen referencia a R. Si este conjunto no es el conjunto vacío, o bien se rechaza la orden borrar como error, o bien se deben borrar las todas las tuplas que hacen referencia a R. La última solución puede llevar a borrados en cascada, dado que las tuplas pueden hacer referencia a tuplas que hagan referencia a R, etcétera.

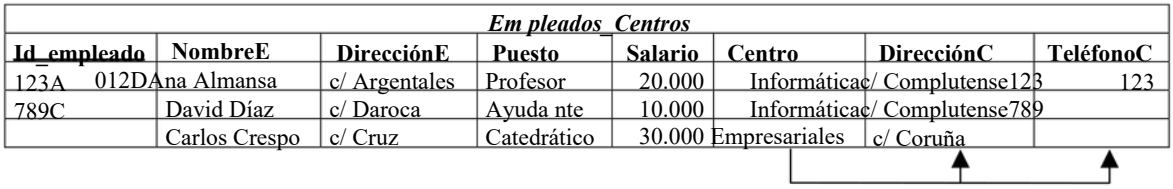
• Actualizar. Hay que considerar dos casos: las actualizaciones de la relación que realiza la referencia (r2) y las actualizaciones de la relación a la que se hace referencia (r1).

- Si se actualiza la tupla *t*2 de la relación r2 y esta actualización modifica valores de la clave externa α, se realiza una comprobación parecida al del caso de la inserción. El sistema debe asegurar que la nueva tupla encuentra sus valores de la clave externa en los de la clave primaria de r1.

- Si se actualiza la tupla t1 de la relación r1 y esta actualización modifica valores de la clave primaria, se realiza una comprobación parecida al del caso del borrado. Si quedan tuplan colgantes (sin correspondencia de clave externa con clave primaria), la actualización se rechaza como error o se ejecuta en cascada de manera parecida al borrado.

2.5.3. Dependencias funcionales

Una dependencia funcional (DF) es una propiedad semántica de un esquema de relación que impone el diseñador. Determina el valor de un conjunto de atributos a partir del valor de otro conjunto de atributos. Por ejemplo, en la siguiente relación se combinan los datos de los empleados, comosu código de identificación y nombre, y de los centros a los que están adscritos, como la dirección y el teléfono.



Em pleados_Centros

Id_empleado	NombreE	DirecciónE	Puesto	Salario	Centro	DirecciónC	TeléfonoC
123A 012D	Ana Almansa	c/ Argentales	Profesor	20.000	Informática	c/ Complutense123	123
789C	David Díaz	c/ Daroca	Ayuda nte	10.000	Informática	c/ Complutense789	
	Carlos Crespo	c/ Cruz	Catedrático	30.000	Empresariales	c/ Coruña	

En este ejemplo se muestra gráficamente que el valor del conjunto de campos DirecciónC y TeléfonoC depende del valor del campo Centro. En concreto, a un centro en particular le corresponden unívocamente una dirección y un teléfono. Es decir, cada vez que aparezca una fila con el valor Informática para Centro, siempre le corresponderá los mismos valores para los campos DirecciónC y TeléfonoC.
Se dice entonces que tanto DirecciónC como TeléfonoC son dependientes funcionalmente de Centro. Por cada fila con un mismo valor de Centro se repiten los valores DirecciónC y TeléfonoC, lo que implica una redundancia de valores no deseable que se estudiará en el tema dedicado a la normalización.

Las dependencias funcionales se denotan de la siguiente forma:

Conjunto de atributos que determinan $\rightarrow$ Conjunto de atributos determinados

En el ejemplo anterior: {Centro} $\rightarrow$ { DirecciónC, TeléfonoC }

2.5.4. Disparadores

Un *disparador* es un mecanismo que se puede usar para implementar restricciones de integridad no soportadas directamente por el SGBD. Un disparador es una orden que el sistema ejecuta de manera automática como efecto secundario de la modificación de la base de datos. Los disparadores

son mecanismos útiles para implementar restricciones de integridad, alertar a los usuarios o para realizar de manera automática ciertas tareas cuando se cumplen determinadas condiciones.

Un ejemplo que se aplica a la imposición de restricciones de integridad sería la implementación de la detección de la violación de las dependencias funcionales.

Otro ejemplo sería enviar indicar cuándo se ha alcanzado un stock mínimo de un producto y que, por tanto, se debe reponer.

Un último ejemplo sería una aplicación bancaria en la que, en lugar de permitir saldos de cuenta negativos, un banco puede tratar los descubiertos dejando a cero el saldo de las cuentas y creando un préstamo por el importe del descubierto. Este préstamo recibe un número de préstamo idéntico al número de cuenta que ha tenido el descubierto.

2.6. Normalización

La traducción del esquema conceptual al lógico no es única. Es necesario contar con una medida de la calidad de la agrupación de los atributos en relaciones. Como herramienta principal se usan las dependencias funcionales para agrupar los atributos en esquemas de relación, que se dice que se encuentran en una determinada forma normal. La forma normal de un esquema de relación determina su grado de calidad con respecto a reducir dos efectos no deseados: la redundancia de datos y las anomalías que produce esta redundancia. Es importante determinar en qué forma normal se encuentra un esquema de relación y el procedimiento, conocido como normalización, para descomponerlo en otros esquemas de relación que se encuentren en formas normales más exigentes.

2.6.1. Redundancia de datos

Un objetivo del diseño de bases de datos relacionales es agrupar atributos en relaciones de forma que se reduzca la redundancia de datos y así el espacio de almacenamiento necesario. Por ejemplo, los siguientes dos esquemas

Empleados(Id_empleado, NombreP, DirecciónP, Puesto, Salario, Centro)

Centros(NombreC, DirecciónC, Teléfono)

contienen la misma información que el siguiente:

Empleados_Centros(Id_empleado, NombreP, DirecciónP, Puesto, Salario,NombreC, DirecciónC, Teléfono)

Empleados					
Id_empleado	**NombreE**	**DirecciónE**	**Puesto**	**Salario**	**Centro**
123A 456B	Ana Almansa	c/ Argentales	Profesor	20.000	Informática
789C 012D	Bernardo Botín	c/ Barcelona	Ad mi nistrati vo	15.000	Matemáticas
	Carlos Crespo	c/ Cruz	Catedrático	30.000	CC. Empresariales
	David Díaz	c/ Daroca	Ayud a nte	10.000	Informática

Centros		
NombreC	**DirecciónC**	**Teléfono**
Informática	Co mpl	123 456
Mate mática s	utense	789
CC. Empresariales	Complutense	
	So mosa gua s	

Empleados_Centros							
Id_empleado	**NombreE**	**DirecciónE**	**Puesto**	**Salario**	**Centro**	**DirecciónC**	**Teléfono**
123A	Ana Almansa	c/ Argentales	Profesor	20.000	Informática	Complutense	123
456B	Bernardo Botín	c/ Barcelona	Administrativo	15.000	Matemáticas	Complutense	456
789C	Carlos Crespo	c/ Cruz	Catedrático	30.000	CC. Empresariales	Somosaguas	789
012D	David Díaz	c/ Daroca	Ayudante	10.000	Informática	Complutense	123

La relación Empleados_Centros presenta redundancia de datos porque se repite para cada empleado la información asociada al centro. Las relaciones con datos redundantes presentan diferentes anomalías de actualización: son las anomalías de inserción, borrado y modificación.

2.6.2. Anomalías de actualización

- Anomalías de inserción. Se produce en dos casos. En primer lugar, cuando se inserta una nueva fila sin respetar las dependencias funcionales. En el ejemplo anterior puede ocurrir si se añade una fila de un empleado adscrito a Informática y con un teléfono distinto de 123. En segundo lugar, la imposibilidad de añadir nuevos datos para el consecuente de la dependencia funcional sin que exista un antecedente para ella. En el ejemplo anterior no se puede dar de alta un centro a menos que exista un empleado destinado en él. Sería necesario dejar valores nulos en la clave (Id_empleado).
- Anomalías de modificación. Se produce cuando se modifican las columnas con datos redundantes de sólo un subconjunto de las filas con el mismo dato. En el ejemplo puede ocurrir cuando se modifica el teléfono de Informática sólo en la primera fila.
- Anomalías de eliminación. Se produce cuando se eliminan todas las filas en las que aparecen los datos redundantes por lo que se pierde los datos de la dependencia funcional. Si se elimina la segunda fila porque el empleado se da de baja, se pierden también los datos del centro.

Las anomalías de actualización aparecen también en los modelos de red y jerárquico, y se resuelven con campos virtuales y tipos de registros virtuales implementados con punteros. Los modelos orientados a objetos evitan el problema mediante la referencia en lugar de la copia.

2.6.3. Formas normales y normalización

La forma normal de una relación se refiere a la mejor forma normal que satisface un esquema de relación indicando así el grado hasta el que se ha normalizado. La indicación del grado de calidad de un esquema de relación se refiere en general en el contexto global del esquema de la base de datos relacional, es decir, en el conjunto de todos los esquemas de relación de la base de datos. Dos propiedades que se deben cumplir para poder asegurarlo son:

- La propiedad de preservación de dependencias, que asegura que las dependencias funcionales originales se mantienen en algún esquema de relación después de la descomposición.
- La propiedad de la posibilidad de reproducir la información de la tabla antes de su descomposición a partir de las tablas resultado de ella.

Las formas normales más habituales, por orden ascendente de exigencia de las propiedades deseadas, son:

Primera (1FN)

Segunda (2FN)

Tercera (3FN)

Boyce/Codd (FNBC)

Hay otras formas normales más exigentes y que se refieren a otras propiedades deseables. Sin embargo, la utilidad práctica de estas formas normales es cuestionable cuando las restricciones en que se basan son difíciles de entender o identificar por los diseñadores y usuarios. Así, en la práctica se usa hasta la forma normal de Boyce/Codd, aunque en general, los diseños prácticos exigen al menos 3FN.

El proceso de asegurar una forma normal para un esquema se denomina normalización.

A continuación se estudiarán las formas normales mencionadas y para cada una de ellas se indicará el procedimiento que asegura que un esquema que no esté en una forma normal determinada se convierta en un conjunto de esquemas que asegure esa forma normal.

2.6.4. Primera forma normal

Actualmente se considera como parte de la definición formal de relación, porque establece que los dominios de los atributos sólo pueden ser atómicos, para evitar atributos multivalorados, compuestos y sus combinaciones. En definitiva evita las relaciones dentro de las relaciones.

Por ejemplo, si se asume que en la entidad Centros, un centro puede tener más de un teléfono, podríamos tener una representación como la siguiente.

Centros		
No mbreC	**DirecciónC**	**Teléfonos**
Informática	Co mpl utense	{123, 321, 213}
Mate mática s	Co mpl utense	{456}
CC. Empresariales	So mosa gua s	{789, 987}

Sin embargo, esto supondría el uso delatributo multivalorado Teléfonos. Hay tres posibilidades de representarlaentidad para satisfacer la primera forma normal:

• Eliminar el atributo Teléfonos y crear unanuevarelación que asocie en cada fila un centro con un teléfono. La clavedelaprimera relación debe formar parte de la clave de la segundarelación. Presenta como inconveniente añadir una nueva relación alesquema de la base de datos y redundancia. Presenta anomalías cuandoseborra un centro y no se borran los teléfonos asociados. La integridadreferencial asegura evitar las anomalías.

Centros	
No mbreC	**DirecciónC**
Informática	Co mp l uten se Co mp l uten se So mosa gua s
Mate mática s	
CC. Empresariales	

Telé fonos	
No mbreC	**Teléfono**
Informática	123
Informática	321
Informática Mate mática s	213
CC. Empresariales	456
CC. Empresariales	789
	987

• Ampliar la clave de la relaciónde manera que incluya al atributo multivalorado. Presenta como inconveniente añadir redundancia que provoca anomalías.

Centros		
No mbreC	**DirecciónC**	**Teléfono**
Informática	Co mpl utense Co mpl utense So mosa gua s So mosa gua s	123
Informática		321
Informática Mate mática s	Co mpl utense Co mpl utense	213
CC. Empresariales		456
CC. Empresariales		789
		987

- Si se conoce la cardinalidad máxima del atributo multivalorado, se pueden crear tantas columnas como la cardinalidad máxima. Presenta como inconveniente el uso de valores Null.

No mbreC	**DirecciónC**	**Teléfono1**	**Teléfono2**	**Teléfono3**
Informática	Co mpl utense Co mpl utense So mosa gua s	123	321	213
Mate mática s		456	Null	Null
CC. Empresariales		789	987	Null

Si el atributo multivalorado es compuesto, como es el caso de representar varias direcciones para un empleado:

Empleados(Id_empleado, NombreE, {Direcciones(Calle, Ciudad, CódigoPostal)})

Esta relación se puede descomponer en dos:

Empleados(Id_empleado, NombreE)

DireccionesE(Id_empleado, Calle, Ciudad, CódigoPostal)

aplicarE ste procedimiento de desanidamiento se puede recursivamente a cualquier relación con atributos multivalorados. teniendo en cuenta que es necesario propagar la clave de la relación original a la clave de la nueva relación, que contiene además la clave que identifica unívocamente al atributo multivalorado.

2.6.5. Segunda forma normal

En el ejemplo a continuación se puede observar que existen anomalías de actualización causadas por las dependencias funcionales DF2 y DF3, ya que como sus antecedentes no son clave, puede haber varias filas con el mismo consecuente. Se usa una notación gráfica para la expresión de las dependencias funcionales. Así:

$DF1 = I\{d_empleado'NúmeroP\} \rightarrow \{Horas\}$

$DF2 = \{Id_empleado\} \rightarrow \{NombreE\}$

$DF3 = \{NúmeroP \rightarrow\} \{NombreP\}$

Personal_Proyectos				
Id_empleado	**NúmeroP**	**Horas**	**No mbreE**	**No mbreP**
123A 012D	P-1	16	AnaAlmansa	Proyecto 1
012D	P-1	8	David DíazProyecto	1
	P-2	4	David DíazProyecto 2	

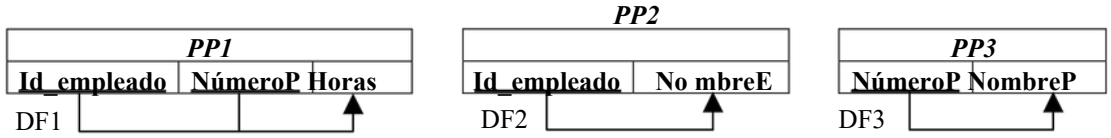


La segunda forma normal evita este tipo de anomalías.

Se dice que una relación está en segunda forma normal si cada atributo que no forme parte de la clave primaria depende funcional y completamente de cada clave.

Un esquema que no se encuentre en segunda forma normal puede traducirse en varios esquemas que sí lo estén. El procedimiento de normalización es crear tantas nuevas relaciones como dependencias funcionales no sean completas.

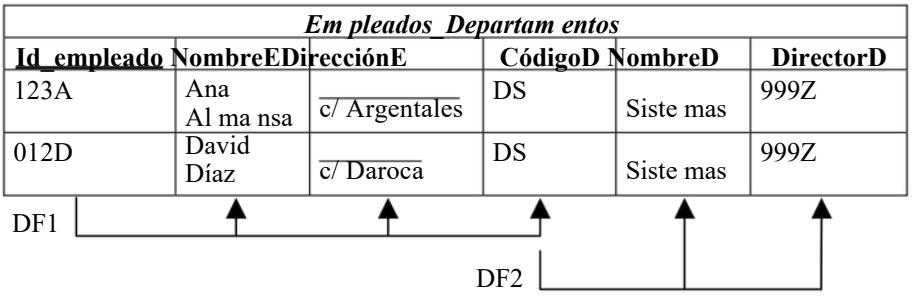
Así, el ejemplo anterior se traduce en:



Hay que observar que este procedimiento asegura que el resultado está, al menos, en segunda forma normal. En particular, y como se puede contrastar con la definición de otras formas normales, el resultado conseguido en este ejemplo se encuentra en FNBC, como se podrá comprobar más adelante.

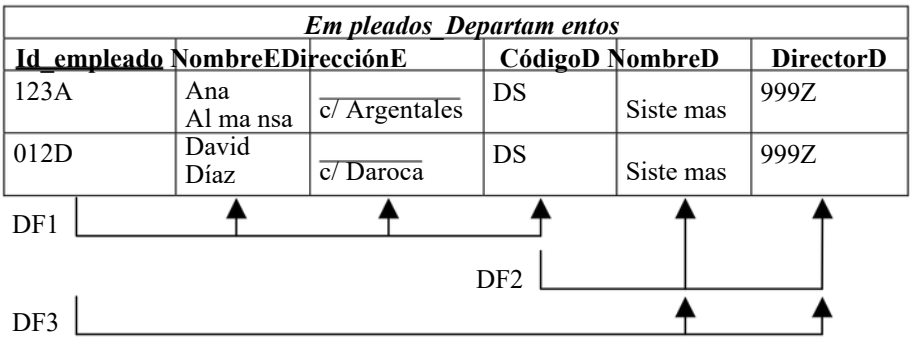
2.6.6. Tercera forma normal

En el ejemplo a continuación se puede observar que existen anomalías de actualización causadas por la dependencia funcional DF2. Sin embargo, este esquema está en segunda forma normal porque los dos últimos atributos, que son los que causan las anomalías, dependen completa (y transitivamente) del único atributo que forma la clave primaria.



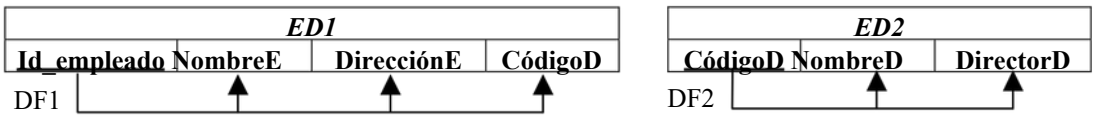
La tercera forma normal se basa en el concepto de dependencia funcional transitiva.Unadependenciafuncional $X \rightarrow Y$esuna dependencia funcional *transitiva* si existe un conjunto de atributos Z que ni forman clave candidata nisonsubconjuntodeningunaclave y ademássse cumple$X \rightarrow Z$ y $Z \rightarrow Y$.

En el ejemplo anterior, DF3 es una dependencia funcional transitiva:



Un esquema está en tercera forma normal si satisface la segunda formanormal y ninguno de los atributos que no forman parte de una clave candidata depende transitivamente de la clave primaria.

El procedimiento para normalizar esta relación consiste en descomponerla en los atributos definidos por la dependencia funcional responsable de la transitividad. En el ejemplo anterior, el resultado de la descomposición es:



El siguiente es un algoritmo que se puede aplicar para calcular estas descomposiciones, donde R es el esquema original a descomponer y F el conjunto de dependencias funcionales que se cumple en R:

D= {R}

1. Encontrar un recubrimiento mínimo T de F

2. for each X → Y∈T, crear en D un esquema Ri con {X ∪{A1}∪...∪{Ak}}

si Ri ⊄Rj ∈ D, dadas las D.F. X →{A1}, ..., X → {Ak}

3. Si ninguno de los esquemas contiene una clave de R, se crea uno nuevo.

Donde el recubrimiento mínimo es el resultado en primer lugar de eliminar todos los atributos de los antecedentes de las dependencias funcionales que sea posible y, en segundo lugar, todas las dependencias

Vfuénacsieo nhatlteps:/ /www.fdi.uqcume. es/profesors/efae rnan/BDSI/appousinbtles. tem

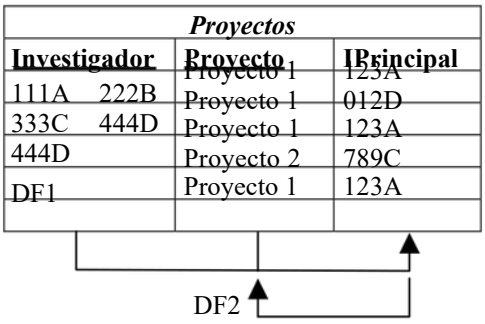
más detalles.

2.6.7. Forma normal de Boyce-Codd

La forma normal de Boyce-Codd (FNBC) se propuso como una forma más simple que la tercera, pero en realidad es más estricta porque cada relación en FNBC está en 3FN pero lo contrario no se cumple.

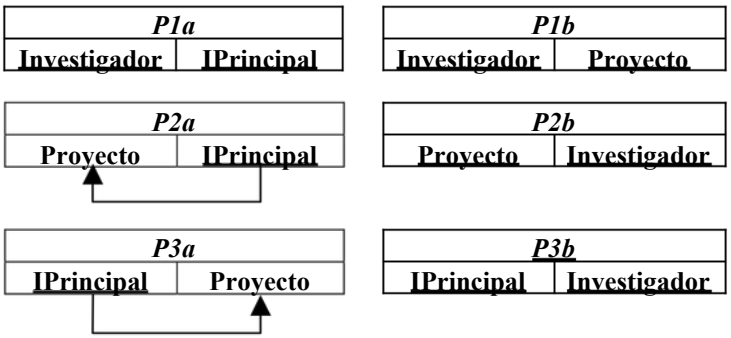
Por ejemplo, considérese la siguiente relación, que está en 3FN pero no en FNBC. La FNBC evita redundancias que la 3FN no puede. En este ejemplo se almacena información de los investigadores participantes en

proyectos, que pueden ser codirigidos, pero los investigadores principales no puedendirigir más de uno.



En este ejemplo, que cumple la 3NF,hayuna anomalía que se debería poder evitar, porque si no se vigilaladependencia funcional DF2 se podría añadir una tupla de manera que unapersona fuese investigadora principal de dos proyectos.

La descomposición del esquemanoes inmediata, hay tres posibilidades:



Todas estas descomposiciones pierden la dependenciafuncional DF1 porque las dependencias funcionales se refieren al contexto local de un esquema, no hacen referencia a esquemas externos. Aúnpeor, las dos primeras generan tuplas incorrectas cuando se trata de recuperar la información de la tabla original. Por ejemplo, en la primera descomposición se pierde la información de los investigadores principales de cada proyecto:

P1a	
Investigador	**IPrincipal**
111A	123A
222B	012D
333C	123A
444D	123A
444D	789C

P1b	
Investigador	**Proyecto**
111A	Proyecto 1
222B	Proyecto 1
333C	Proyecto 1
444D	Proyecto 1
444D	Proyecto 2

P1a ? *P1b*		
Investigador	**Proyecto**	**IPrincipal**
111A	Proyecto 1	123A
222B	Proyecto 1	012D
333C	Proyecto 1	123A
444D	Proyecto 2	123A
444D	Proyecto 2	789C
444D	Proyecto 1	123A
444D	Proyecto 1	789C

Por tanto, ninguna de estas dos últimas descomposiciones es aceptable.

En la práctica, la mayoría de los esquemas que están en 3NF lo están también en FNBC.

2.6.8. Desnormalización para el rendimiento

A veces los diseñadores de bases de datos escogen un esquema que tiene información redundante; es decir, que no está normalizada. Utilizan la redundancia para mejorar el rendimiento para aplicaciones concretas. La penalización sufrida por no emplear un esquema normalizado es el trabajo extra (en términos de tiempo de codificación y de tiempo de ejecución) de mantener consistentes los datos redundantes.

Por ejemplo, supóngase que hay que mostrar el nombre del titular de una cuenta junto con el número y el saldo de su cuenta cada vez que se tiene acceso a la cuenta. En el esquema normalizado esto exige una reunión de *cuenta* con *impositor*.

Una alternativa para calcular la reunión sobre la marcha es almacenar una relación que contenga todos los atributos de *cuenta* y de *impositor*. Esto hace más rápida la visualización de la información de la cuenta. No obstante, la información del saldo de la cuenta se repite para cada persona titular de la cuenta, y la aplicación debe actualizar todas las copias, siempre que se actualice el saldo de la cuenta. El proceso de tomar un esquema normalizado y hacerlo no normalizado se denomina desnormalización, y los diseñadores lo utilizan para ajustar el rendimiento de los sistemas para dar soporte a las operaciones críticas en el tiempo.

Una alternativa mejor, soportada hoy en día por muchos sistemas de bases de datos, es emplear el esquema normalizado y, de manera adicional, almacenar la reunión en forma de vista materializada. (Una vista materializada es una vista de la base de datos cuyo resultado se almacena en la base de datos y se actualiza cuando se actualizan las relaciones

utilizadas en la vista). Al igual que la desnormalización, el empleo de las vistas materializadas supone sobrecargas de espacio y de tiempo; sin embargo, presenta la ventaja de que conservar la vista actualizada es labor del sistema de bases de datos, no del programador de la aplicación.

2.7. Normativa de denominación

La normativa de denominación es una colección de reglas que permite asignar nombres a identificadores y objetos. El objetivo es que los nombres que se elijan indiquen de forma lo más clara posible el significado del elemento al que se refiere el nombre. Cada empresa suele usar una normativa diferente y más o menos complicada. A veces se tienen manuales de treinta páginas que la describen. En este documento se aconsejarán algunas reglas que se pueden usar para establecer una normativa de denominación.

2.7.1. Identificadores

Los identificadores (o nombres que se usan para designar los elementos de una base de datos) se construyen generalmente con letras y números. En muchos SGBD no se distinguen mayúsculas de minúsculas, pero su uso nos puede ayudar a hacer más legibles los identificadores. Cuando los identificadores tienen más de una palabra hay dos alternativas que se usan habitualmente:

- Separar cada palabra por un carácter de subrayado, como en Nombre_del_paciente.
- Separar cada palabra poniendo la primera letra de cada una en mayúsculas, como en NombreDelPaciente.

Otra alternativa final, menos usada, es usar espacios en blanco para la separación, como en Nombre del paciente. Algunos SGBD tienen problemas con estos espacios en blanco y por eso no se usan habitualmente. Incluso tienen problemas con algunos caracteres como las vocales acentuadas y las eñes, por lo que en general se evitan. En Access, sin embargo, no se tienen estos problemas y hemos seguido esta normativa de denominación de los identificadores.

2.7.2. Tablas

Las tablas representan entidades y sus nombres deberían describir las entidades que representan. Por ejemplo, Pacientes sería un identificador que describe a una tabla que contiene información sobre las entidades Pacientes. Además se escribe en plural porque el tipo de entidad contiene un conjunto de entidades (la tabla contiene varios pacientes en general).

Algunas tablas, sin embargo, no presentan entidades. Pueden representar relaciones entre entidades. Por ejemplo, la relación entre Personas y Teléfonos se puede denominar Tiene teléfono. Cuando no se pueda encontrar un identificador adecuado para una relación siempre se

puede escribir un identificador con los dos nombres de las tablas, como PersonasTeléfonos.

Reglas básicas de denominación de tablas:

• Seleccionarnombres de tablasbasados en losnombresposiblesparalas entidades involucradas.

• Usar sustantivos para los nombres de las tablas.

• Asegurarse dequelos nombrestienen un sentido intuitivo en lacultura de quienes utilizan la base de datos.

• Expresarlas relaciones capturadaspor las tablas mediante la vinculación de los nombres de las entidades vinculadas por la tabla.

En las tablas se tiene que denominar a las columnas. Las columnas representan elementos discretos de información sobra la entidad nombrada por la tabla. Normalmente tampoco representan relaciones. Debido a estos hechos el nombre de una columna debería ser un sustantivo que nombre el elemento de información que representa. Debería ser un sustantivo que reflejara la forma que los usuarios hablan sobre el elemento de información y debería reflejar características sobresalientes sobre el elemento de información. Por ejemplo, se usará Nombre como identificador del atributo nombre de pila de un paciente.

2.7.3. Restricciones

Las restricciones se pueden denominar de formas autointerpretativas. Hay que utilizar una abreviatura de dos letras para identificar la naturaleza de la restricción:

• CP (o PK en inglés, primary key) para clave principal

• IR (o RI en inglés, referential integrity) para integridad referencial

• CO (o CK en inglés, check) para la de comprobación

• UN para la de unicidad.

Después hay que utilizar el nombre de la tabla a la cual se aplica la restricción como segundo elemento del nombre. Una restricción de clave principal para la tabla Médicos se debería nombrar CP_Médicos.

En el caso de claves externas, donde están involucradas dos tablas, hay que poner el nombre de la segunda tabla como tercer elemento para el nombre de la restricción. Una clave externa para las tablas Médicos y Especialidades tendría el nombre IR_Médicos_Especialidades. Cuando están involucradas dos tablas hay que hacer que el segundo elemento del nombre sea la tabla con la clave externa y que el tercer elemento sea la tabla donde reside la clave principal.

Finalmente, para las restricciones de comprobación, que tienen un papel funcional, hay que agregar un elemento final al nombre que describe su función. Una restricción de comprobación que garantice el formato de un

número telefónico, por ejemplo, podría tener este nombre: CO_Médicos_FormatoCódigo.

2.7.4. Controles

Cada tipo de control se debería denominar con una indicación del tipo de control, anteponiendo a un nombre descriptor un prefijo que indique el tipo, como se propone en la siguiente tabla.

Control	Prefijo	Ejemplo
Cuadro de texto	txt (acrónimo de text, texto en inglés)	txtEntrada
Etiqueta	lbl (acrónimo de label, etiqueta en inglés)	lblEtiqueta
Cuadro combinado	cmb (acrónimo de combo box, cuadro combinado en inglés)	cmbProvincias
Botón de comando	but (acrónimo de button, botón en inglés)	butCancelar
Grupo de opciones (marcos)	fra (acrónimo de frame, marco en inglés)	fraOperaciones
Botón de opción	opt (acrónimo de option, opción en inglés)	optCubo
Botón de selección	chk (acrónimo de check, verificación en inglés)	chkVerde
Lista	lst (acrónimo de list, lista en inglés)	lstPacientes

2.7.5. Variables

Cada variable se debería denominar con una indicación del tipo de la variable, anteponiendo a un nombre descriptor un prefijo que indique el tipo, como se propone en la siguiente tabla.

Tipo de variable	Prefijo	Ejemplo
Byte	byt	bytEdad
Boolean	boo	booCasado

Integer	int	intPersonas
Long	lng	lngLlamadas
Single	sgl	sglRaíz
Double	dbl	dblSuperficie
Currency	cur	curEuros
Date	dat	datFecha
Object	obj	objObjeto
String	str	strNombre
Variant	var	varElemento

2.7.6. Objetos de la base de datos

Cada objeto de labase de datosse debería denominar con una indicación del tipo de objeto, anteponiendo a un nombre descriptor un prefijo que indique el tipo, como se propone en la siguiente tabla.

Tipo de objeto	**Prefijo**	**Ejemplo**
Tabla	tbl (acrónimo de table, tabla en inglés)	tblPacientes
Formulario	frm (acrónimo de form, formulario en inglés)	frmMédicos
Informe	rep (acrónimo de report, informe en inglés)	repPacientes
Consulta	qry (acrónimo de query, consulta en inglés)	qryGastoTrimestral